

Eval-driven development: Lessons from evaluating GenAI at scale

How Airbnb teams build trustworthy Generative AI products by treating evaluation as a first-class engineering discipline; not an afterthought.

By: Rohit Girme, Dan Miller, Mia Zhao, Lifan Yang, Clint Kelly.

Introduction

Generative AI breaks a lot of the assumptions that used to hold true for software testing. Unlike traditional software, LLM outputs are non-deterministic, and “*correct*” is subjective. Because so much judgment is involved, you often need an AI to evaluate an AI, which introduces its own potential failure modes. Making matters more complicated, a single interaction with an LLM can chain retrieval, reasoning, tool calls, and generation, each of which can fail independently.

working, guiding where we improve the product next.

Each product team may have its own evaluation criteria, process, workflows, etc. However, these are built on top of some common foundations and principles. An infrastructure team provides tooling and best practices, incorporating learnings across domains so that they are shared with everyone building products at Airbnb.

In this article, we wanted to share some of these best practices and learnings with the broader engineering community. Please note that the recommendations here are not intended to be prescriptive; there is no one-size-fits all approach when it comes to running evals.

1. Foundation

Evaluating LLM-based systems is challenging work, and this should be planned for at the outset. Without a deliberate strategy, three things tend to happen:

- False confidence: A generic “helpfulness” metric scores well, you ship, but it didn’t capture the failure mode people actually hit.
- Undetected regressions: A prompt change subtly degrades a dimension you weren’t measuring.
- Wasted effort: You build a scaled eval pipeline for metrics that don’t correlate with outcomes.

Expect to spend a meaningful share of your total project effort on evaluation. This is not unnecessary overhead, it’s how you build products that actually work.

1.1 The one rule

When in doubt, look at your data. Manually reviewing your data and building an intuition for what counts as success is always the starting point we recommend to teams. Build your prototype, and run it through 100 examples (synthetic is fine). Then *read the outputs*. Read the traces and find the model’s mistakes. Categorize them and build an eval.

This single habit will do more for your product quality than any framework, tool, or methodology in this document.

Formalized, that habit becomes **eval-driven development (EDD)**, the GenAI analogue of test-driven development. Rather than predicting every failure upfront, EDD builds the infrastructure and habits to **discover, encode, and continuously test** for failure modes as they appear. It also forces stakeholders to externalize what “good” means, which shapes the product roadmap.

Five principles anchor EDD:

1. Define goals and gates upfront. What are you optimizing for? What must be true before you ship? These answers may not be clear right away; you might discover them as part of your data exploration.
2. Let real errors guide your metrics. Co-develop them with cross-functional partners based on observed failures. Don't invent them in a vacuum.
3. Keep your evaluator set small and sharp. 3–5 well-calibrated LLM-as-judge evaluators beat 20–30 noisy ones. Each should target one specific correctness dimension.
4. Appoint a decision-maker. While what constitutes correctness should be a team discussion, people will sometimes disagree. Include a final (human) decision-maker who makes the ultimate call on what constitutes good vs. bad system behavior.
5. Collaborate continuously. Have your product partner regularly answer: “Is X better or worse than Y?” and “What’s actually wrong with this output?”

2. The three evaluation methods

Every evaluation you run will use one or a combination of these three methods.

Layer 1: Programmatic checks (fast, low resource — catches obvious failures)



Layer 2: LLM-as-a-Judge (nuanced - catches quality issues)



Layer 3: Human evaluation (high resource - validates edge cases, calibrates the stack)

Deterministic, code-based checks that don't require an LLM call should be your first filter, catching obvious failures before you send anything to a judge or human labeler.

Check type	Examples	When to use
Format validity	Does the output parse as valid JSON? Does it match the expected schema?	Always. Especially for structured outputs feeding downstream systems
Length & presence	Is the response empty? Suspiciously short or long?	Always. Catches obvious generation failures
Keyword/regex	Forbidden words, required disclaimers	Safety, compliance, brand voice
Classical ML metrics	Precision, recall, F1, exact match	Classification, entity extraction, retrieval
Semantic similarity	Cosine similarity against reference embeddings	When you have gold-standard reference answers

- ✔

Do: Use structured outputs (JSON schemas) to ensure strict typing.
- ✗

Don't: Rely on prompt instructions alone to format data. This breaks downstream data pipelines.

2.2 LLM-as-judge (Virtual judges)

Use a stronger LLM to evaluate another LLM's output against a carefully designed rubric. This is how you assess nuanced qualities e.g. tone, coherence, faithfulness, relevance, at a fraction of the resources needed for human evaluation.

	"helpfulness, safety, AND tone" is impossible to debug or calibrate.	prompts for each dimension, with detailed criteria and instructions. At Airbnb, it's normal for an eval pipeline to have 1–7 virtual judges. For our AI customer support product, for example, we have separate virtual judges to measure different dimensions of performance (e.g., correctness), risk (e.g., sensitive data handling), and product alignment (e.g., style and tone). Not every dimension needs a frontier model; to improve efficiency, test whether a smaller model suffices.
Different judge than generator	Models favor their own style (self-bias).	Always use a different model family or version.
Few-shot examples	Without examples, the model's definition of "good" won't match yours.	Include 2–3 examples that meet your judge's criteria, and 2–3 examples that don't meet it. Caveat: Over-reliance on examples risks overfitting. A well-articulated rubric can outperform examples alone.
Explicit output schema	Free-form responses break automated pipelines.	Define the exact JSON structure; include a reasoning field so the LLM justifies its decision. Note that while JSON is the most common, open-source inference frameworks may support additional structured output formats.
Clear scoring	Either a binary score (pass/fail) or Likert score output are needed to calibrate and aggregate model responses.	Default to binary scoring, and use Likert when you need a gradient signal (e.g., severity). Always define what each point means.

consistently, an LLM certainly can't.

Here is a simplified example of a single virtual judge's rubric:

Score the readability of listing explanations. A good explanation sounds like a friendly travel agent: warm but professional, simple, natural, grammatically complete.

Score 1 if it reads cleanly.

Score 0 if it has ANY of these problems:

- Tone: too formal/jargony, too casual ("awesome vibes"), too salesy ("amazing!"), or robotic.
- Internal terms: never use internal terminology.
- Formatting: no quotation marks, no bullets, no fragments. End every explanation with a period - never "!" or "?".
- Grammar: use articles/determiners/prepositions for natural flow ("this home has a pool", "close to downtown"). In a series, use the article once then drop it: "a backyard, grill, and kitchen" - not repeated, not omitted entirely.
- Complexity: plain words over jargon ("pool" not "aquatic recreation area"; "near" not "proximate").

Examples:

- "Host mentions a pool and hot tub available near downtown." → 1
- "The listing mentions a pool!" → 0 (internal term "listing"; ends in "!")
- "This domicile encompasses aquatic amenities." → 0 (complex words; jargon)

Return ONLY:

```
{  
"reason": "<list of [error_type, explanation] tuples as a string, or []>",  
"score": <1 or 0>  
}
```



A virtual judge that hasn’t been calibrated is worse than no judge at all, because it gives you false confidence. Here are the calibration steps we recommend:

1. Create a golden dataset of 50–100 examples. This MUST include bad examples (not just good ones).
2. Run your virtual judge against the golden set.
3. Measure agreement. Target percentages in the high 80s-90s. Possible options to measure disagreement are Cohen’s kappa or Krippendorff’s alpha. (Perfect agreement isn’t achievable — even humans disagree.)
4. Analyze disagreements. Refine the prompt and update your few-shot examples. Then re-run the loop until you hit the target agreement.
5. Recalibrate periodically as failure modes evolve.

2.3 Human evaluation

Human judgment remains the gold standard for ground truth, high-stakes domains, and resolving disagreements between automated evaluators.

Scenario	Why humans are essential
Ground truth creation	You can't measure accuracy without knowing correct answers, so human labelers can be used to create “golden sets.” You can achieve this either through general annotators or subject matter experts (e.g., employees with customer support expertise for a guest facing support use case). You can also combine human inputs to achieve a ground truth via a majority vote.
Safety & high-stakes	High risk scenarios implicating things like trust or discrimination require human verification as a mandatory gate.
Nuance & creativity	Humans still outperform LLMs on humor, brand voice, and "delight."
Evaluator disagreement	When your virtual judge and programmatic checks disagree, a human breaks the tie.

Overall, the rule of thumb is to start with 20–100 rows labeled by subject-matter experts. Move to a scaled annotation workforce only when the rubric is rock-solid and volume is the bottleneck.

And if your experts disagree on a label, **stop**. Solve human disagreement before automating anything.

				Compare model output with output from a frontier model. Then leverage standard (programmatic) metrics like precision, recall, and F1 score to compare the predicted and reference outputs. During the development cycle, programmatic and virtual judges can aid in velocity while maintaining quality. Also note, caching of results can assist in keeping minimizing resource use. Only re-evaluate new or changed rows.
Pre-release gate	✅ Baseline	✅ Comprehensive	✅ Critical cases	For another search related product, we used 7 VJs for evaluation. Each VJ evaluated a single metric (e.g., whether the model response had the appropriate voice and tone, whether it was representing relevant details correctly, etc.). Before releasing to production, we leveraged a combination of all three techniques to catch edge cases, similar to end to end integration testing by humans when deploying code.
Production monitoring	✅ Primary	✅ Flagging	✅ Escalation	For a production AI customer support agent, at regular intervals, we send product traffic through risk and performance VJs and get a score out of each VJ. These are aggregated into scores for that interval and monitored via dashboards. Answers flagged by VJs can be sampled and sent for human review, subject to our strict privacy-gated protocols and purpose-limitation controls designed to preserve user anonymity.

				~ suspected risk cases. They together compose a standard passive monitoring flow.
--	--	--	--	--

3. Evaluating agentic systems

Agentic systems involve multi-step reasoning, tool calling, branching logic and intermediate state transitions. Evaluating only the final output is insufficient: a correct final answer can mask a broken reasoning path, wrong tool parameters, or an inefficient trajectory.

Therefore, you will need to evaluate across three layers:

Layer	What you evaluate	Example
Step-level	Was each tool call, retrieval, or reasoning step correct?	"Did the agent call the booking API with the correct date parameters?"
Trajectory-level	Was the overall path reasonable and efficient?	"Did the agent take 2 steps or 12 steps to resolve a simple question?"
Session-level	Did the full interaction achieve the person's goal?	"Was the issue resolved?"

To achieve this, you can take advantage of the fact that an agent generally pushes traces and spans under an application root. This contains information about the type of agent, the sub agent if invoked, input/output of the agent, tools invoked if any, and more. These traces can be written out to an observability platform or persistent storage.

Then, you can use DFS or another type of tree traversal to reconstruct the trace in memory. This lets you ensure certain subagents were invoked at the right time, the agent called the right tools, etc. And you can scope your evaluation to specific agents/subagents.

4. A practical walkthrough

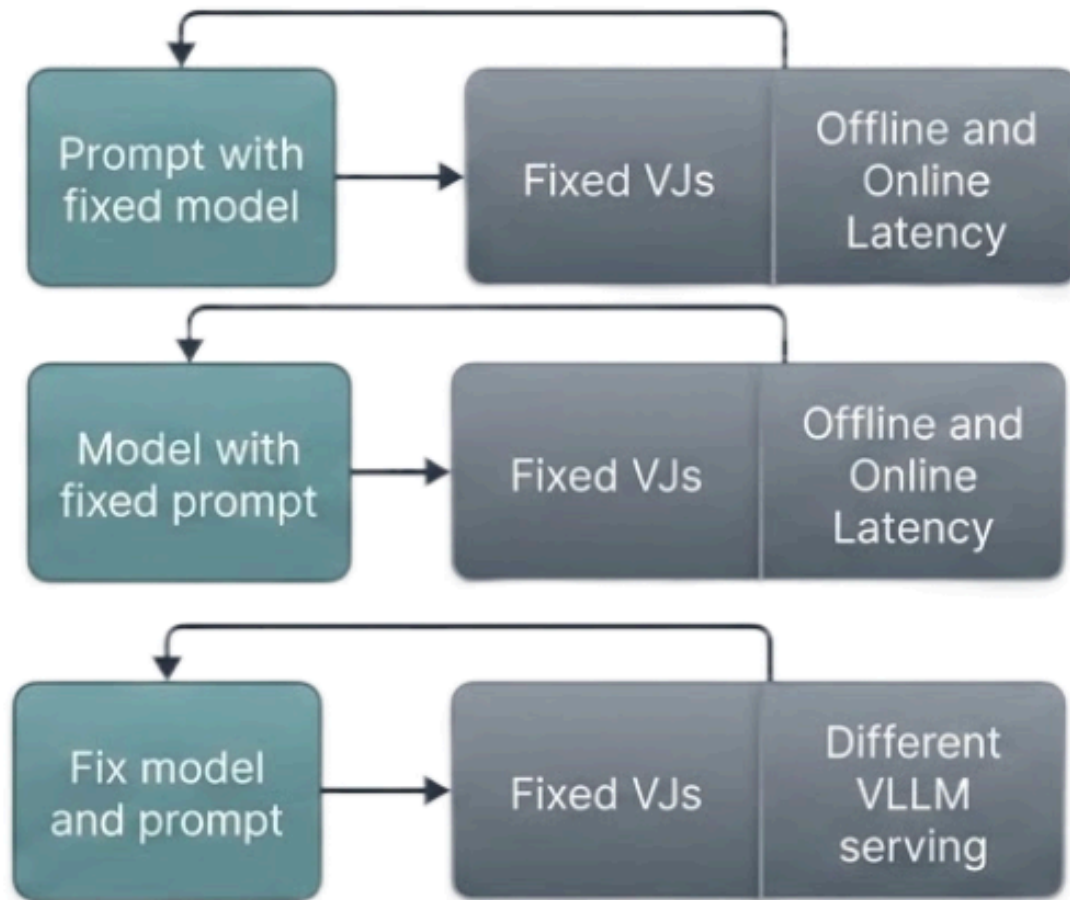
Here’s what the full process looks like end-to-end, using a fictionalized and simplified version of a real use case.

Step 1: Explore & discover. Run 100 inputs through your prototype and read every output. You find: 15 responses generated policy details not in the source documents (*faithfulness issue*); 8 correct but too verbose (*conciseness*); 5 refused valid questions (*over-refusal*); 3 had broken JSON (*format*).

Step 2: Build evals. Add programmatic checks for JSON validity and length bounds. Write a virtual judge for faithfulness (separate prompt, different model, chain-of-thought) and another for conciseness. Have your PM or subject matter expert label 60 examples, including failures, as a golden set.

Step 3: Calibrate & iterate. Your faithfulness virtual judge agrees with the PM 78% of the time. Not good enough. Analysis reveals the judge is penalizing accurate paraphrases as “unfaithful.” Update the rubric and add few-shot examples. Agreement jumps to 88%. Improve the retrieval step; faithfulness failures drop significantly.

NOTE: Here, we find that when iterating on models and prompts, it's best to fix one variable at a time. First fix the model and vary the prompt, then fix the prompt and vary the model, then fix both and vary the serving configuration. At each stage, virtual judge results narrow the candidate pool. Then, you can improve the virtual judge(s) using samples from the top candidates. The evaluators and the candidates sharpen each other until both stabilize.



Step 4: Scale & monitor. Scale evaluation across 5,000 examples. Set up production monitoring: sample 5% of live de-identified traffic daily, run programmatic checks + virtual judges, and surface flagged outputs for human review. A weekly PM review closes the loop, with new failure modes introducing new evals and subsequent system improvements.

NOTE: We sample live traffic continuously using privacy-preserving techniques. All data undergoes robust de-identification prior to human review, and usage is strictly purpose-limited to safety and quality assurance, aligning with Airbnb Privacy Principles.

Key takeaways

1. Look at your data. Read outputs and traces before building anything else.

4. One evaluator per dimension. No “God evaluators.”
5. Calibrate to high 80s-90s% agreement before trusting your Virtual Judge at scale.
6. Use all three methods. Programmatic, Virtual Judge, and human as layered defenses.
7. Include bad examples in your Gold Set. You can’t test discernment without them.
8. Evaluate the system, not just the model. Test retrieval, tool calls, the full pipeline. For agents, evaluate the trajectory, not only the final answer.
9. Mirror evals in production. Pre-production metrics are not one-and-done.
10. Evaluation is a team sport. Evaluation is about shaping what product success looks like, and that takes contributions from many people. The teams that succeed with AI aren’t the ones with the best models, they’re the ones with the best communication and clearest product vision.

If this type of work interests you, check out some of our [related positions](#)!

Acknowledgments

We would like to thank Tania Myronivska, Haozhen Ding, and Sebastian Wickenburg for their thoughtful feedback and contributions to this guide, Jisheng Liang and John Hewson for their guidance and insights, and Min Yi and Yi Li for their constant support.

We would also like to thank Evelyn Xu for their support in authoring this post during their time at Airbnb.

All product names, logos, and brands are property of their respective owners. All company, product, and service names used in this website are for identification purposes only. Use of these names, logos, and brands does not imply endorsement.

All product names, logos, and brands are property of their respective owners. All company, product and service names used in this website are for identification purposes only. Use of these names, logos, and brands does not imply endorsement.

